

CSA1006-SOFTWARE ENGINEERING

C. Pavani (192524260)

Unit 2

Assessment

Problem 4: API-First Development for a Hotel Booking Platform

Scenario

A software development company is building a Hotel Booking Platform where the Frontend, Backend, and Mobile App teams need to work simultaneously. To avoid delays and improve collaboration, the Project Manager has decided to follow an API-First Development approach using OpenAPI (Swagger).

Understanding the Problem Statement

Problem Explanation

In a traditional development process, backend APIs are developed first, and frontend/mobile teams must wait until they are completed. This causes delays, frequent requirement changes, and communication issues.

Using API-First Development, the API is designed and documented before coding begins. All teams can work in parallel using the API specification.

Objectives

- Design APIs before implementation.
- Enable parallel development among teams.
- Reduce integration issues.
- Improve communication using Swagger documentation.
- Ensure consistent API standards.

Scope

- Hotel Search API
- Room Availability API
- Booking API
- Payment API
- User Authentication API
- API Documentation using Swagger
- Docker Container Deployment

1. API-First Development

What is API-First Development?

API-First Development is a software development approach where the API contract is designed before writing any application code.

The API documentation acts as a blueprint for developers.

Why API-First is Suitable?

1. Parallel Development

Frontend, Backend, and Mobile developers can work simultaneously using the API documentation.

2. Better Communication

Swagger provides a common understanding of APIs.

Everyone follows the same specification.

3. Faster Development

No need to wait for backend completion.

Mock APIs can be used during development.

4. Reduced Errors

All request and response formats are predefined.

Less chance of integration failures.

5. Easy Maintenance

Any API change is documented immediately.

Developers know exactly what changed.

Advantages of API-First Development

Advantage	Description
Faster Development	Teams work simultaneously
Better Collaboration	Common API documentation
Easy Testing	Mock APIs available
Improved Consistency	Standard API format
Easier Maintenance	Version-controlled APIs

2. Swagger (Open API) API Design

Sample API

Room Availability Check

GET /api/v1/rooms/availability

Description

Returns available hotel rooms for the selected location and date.

Request Parameters

Parameter	Type	Required	Description
city	String	Yes	Hotel Location
checkIn	Date	Yes	Check-in Date
checkOut	Date	Yes	Check-out Date
guests	Integer	Yes	Number of Guests

Example Request

GET /api/v1/rooms/availability

city=Chennai

checkIn=2026-08-10

checkOut=2026-08-12

guests=2

Successful Response (200 OK)

```
{  
  "hotelName": "Grand Palace Hotel",  
  "roomType": "Deluxe",  
  "pricePerNight": 4500,  
  "availableRooms": 12,  
  "rating": 4.7  
}
```

Error Response (404)

```
{  
  "message": "Rooms Not Available"  
}
```

Swagger Documentation Outline

Field	Value
Method	GET
Endpoint	/api/v1/rooms/availability

Field	Value
Description	Check room availability
Parameters	city, checkIn, checkOut, guests
Response	JSON
Authentication	JWT Token

3. API Versioning Strategy

Why Version APIs?

Applications continue to evolve.

If APIs change suddenly, old mobile apps or websites may stop working.

Versioning allows multiple API versions to coexist.

Versioning Method

URI Versioning

/api/v1/rooms

/api/v2/rooms

This is the most widely used versioning strategy.

Other Versioning Methods

Method	Example
URI Versioning	/api/v1/rooms
Header Versioning	Accept-Version:v2
Query Parameter	?version=2

Recommended Strategy

Use URI Versioning

Example:

GET /api/v1/rooms

GET /api/v2/rooms

Benefits:

- Easy to understand
- Simple to maintain
- Supports backward compatibility

4. Docker Containerization

What is Docker?

Docker packages an application with all its dependencies into a lightweight container.

The application runs the same on every computer or server.

Docker Architecture

Developer

|



Docker Image

|



Docker Container

|



Cloud Server

|



Hotel Booking API

Benefits of Docker

1. Consistent Environment

Application behaves the same in Development, Testing, and Production.

No "works on my machine" problems.

2. Faster Deployment

Containers start within seconds.

Suitable for cloud deployment.

Additional Benefits

- Easy Scaling
- Lightweight
- Better Resource Utilization
- Faster Recovery
- Simplified Maintenance

Workflow of API-First Development

Requirement Analysis

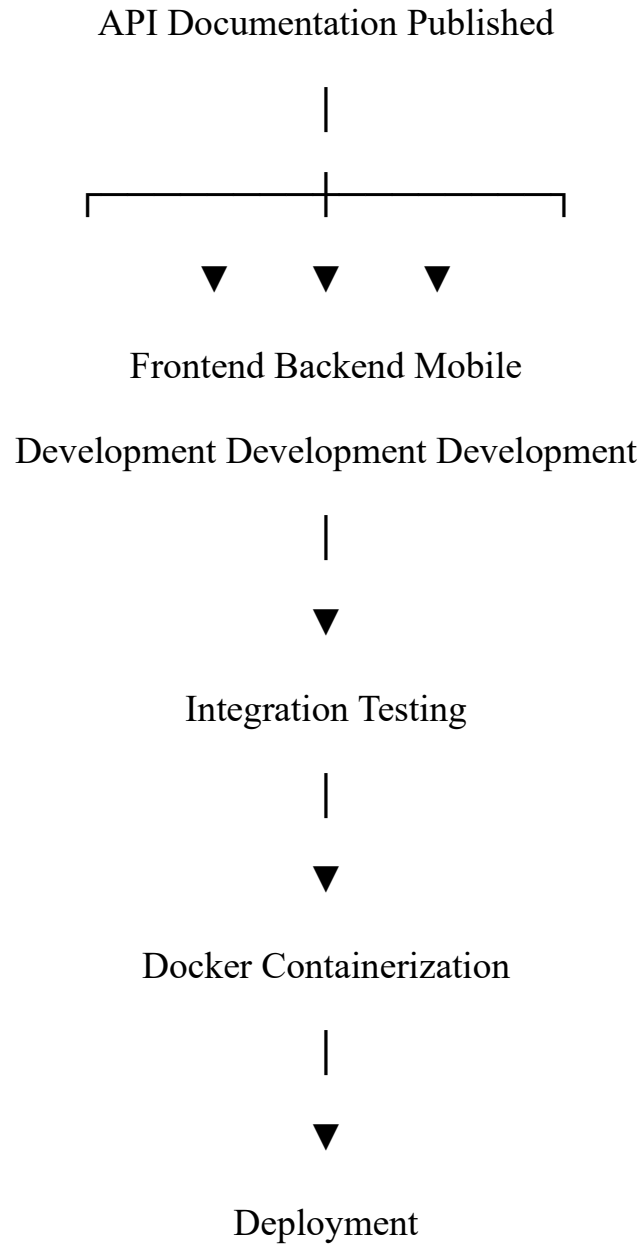
|



API Design (Swagger/OpenAPI)

|





Real-World Example

Hotel Booking Process

1. User opens Hotel Booking App.
2. Mobile app calls:

GET /api/v1/rooms/availability

3. Backend checks database.
4. Available rooms returned.
5. User books room.
6. Booking API stores reservation.
7. Payment API processes payment.
8. Confirmation sent to user.

Expected Outcomes

Benefits

- Parallel development
- Reduced development time
- Standardized API documentation
- Easy integration
- Better communication
- Scalable architecture
- Faster deployment using Docker

Challenges

- API design requires careful planning.

- Documentation must be updated regularly.
- Version management increases maintenance.
- Docker requires initial learning.

Future Enhancements

- GraphQL API Support
- Kubernetes Deployment
- AI-based Hotel Recommendation
- Multi-language API
- Cloud-native Microservices
- API Gateway Integration

Output:

The screenshot displays the Swagger UI for the 'Hotel Booking API v1'. The left sidebar lists various API endpoints under categories like 'Rooms', 'Booking', 'Payments', 'Users', and 'Authentication'. The main panel shows the 'Room Availability Check' endpoint (GET /rooms/availability). It includes a table of parameters: 'city' (string, required), 'checkIn' (string, required), 'checkOut' (string, required), and 'guests' (integer, required). Below the parameters, the 'Responses' section shows two possible outcomes: a '200 Successful Response' with a JSON example containing hotel details and room availability, and a '404 Rooms Not Available' response with a JSON example containing a message. The interface also features an 'Authorize' button and a 'Try it out' button.

Hotel Booking API
v1 OAS 3.0
API documentation for the Hotel Booking Platform.

Room Availability Check
GET /rooms/availability Check available rooms for a hotel based on location and dates.

Parameters

Name	Description	Required	Schema
city	Hotel location/city name	true	string
checkIn	Check-in date (YYYY-MM-DD)	true	string(\$date)
checkOut	Check-out date (YYYY-MM-DD)	true	string(\$date)
guests	Number of guests	true	integer(\$int32)

Responses
Response content type: application/json

200 Successful Response
Example Value | Schema

```
{  "hotelName": "Grand Palace Hotel",  "roomType": "Deluxe",  "pricePerNight": 4500,  "availableRooms": 12,  "rating": 4.7}
```


Content type: application/json

404 Rooms Not Available
Example Value | Schema

```
{  "message": "Rooms Not Available"}
```


Content type: application/json

Conclusion

The API-First Development approach enables frontend, backend, and mobile teams to work in parallel by designing APIs before implementation. Using OpenAPI (Swagger) provides clear and standardized API documentation, reducing integration issues and improving collaboration. Implementing API versioning ensures backward compatibility, while Docker containerization provides consistent environments, faster deployment, and scalability. This modern approach helps deliver a reliable, maintainable, and efficient hotel booking platform that supports future growth and seamless team collaboration.

